# Library Management System – User & Developer Guide

## Student Information

**Name:** Derick Burjack Cardoso

**Student ID:** 6300410

## 1. Overview

This Library Management System is a Java-based application that allows users to manage library items such as books, DVDs, and magazines. It supports borrowing, returning, searching, reporting, and data persistence using CSV files.

## 2. User Guide

### 2.1 User Types

#### Student

- Can borrow **books only**
- Maximum of **5 borrowed books**

#### Teacher

- Can borrow **any item type**
- Maximum of **10 items**

#### Admin

- Can generate system reports

```java
Admin admin = new Admin( name: "Rick");
Book book1 = new Book( title: "JAVA GUIDE", Item.Status.IN_STORE,
        ISBN: "978-3-16-148410-0",  author: "Kim",  genre: "guide");
Book book2 = new Book( title: "JAVA GUIDE2", Item.Status.LOST,
        ISBN: "978-1-861-97271-2",  author: "Kim",  genre: "guide");
DVD dvd1 = new DVD( title: "FNF", Item.Status.BORROWED,  director: "Mark",  minutes: 78);
DVD dvd2 = new DVD( title: "FNF2", Item.Status.IN_STORE,  director: "Mark",  minutes: 90);
Magazine magazine1 = new Magazine( title: "BAt", Item.Status.LOST,  issueNumber: 1,  publisher: "fell");
Magazine magazine2 = new Magazine( title: "BAt2", Item.Status.BORROWED,  issueNumber: 2,  publisher: "fell");
admin.reportItems();
```

```
In Store Items [DVD(super=Item(id=0004, title=FNF2, status=IN_STORE), director=Mark, minutes=90), Book(super=Item(id=0001, title=JAVA GUIDE, status=IN_STORE), ISBN=978-3-16-148410-0, author=Kim, genre=guide)]
Borrowed Items: [Magazine(super=Item(id=0006, title=BAt2, status=BORROWED), issueNumber=2, publisher=fell), DVD(super=Item(id=0003, title=FNF, status=BORROWED), director=Mark, minutes=78)]
Lost Items [Magazine(super=Item(id=0005, title=BAt, status=LOST), issueNumber=1, publisher=fell), Book(super=Item(id=0002, title=JAVA GUIDE2, status=LOST), ISBN=978-1-861-97271-2, author=Kim, genre=guide)]
```

- Can back up users and items

## 2.2 Borrowing Items

To borrow an item:

1. Search for an available item (IN_STORE status)
2. Select the item

```java
Student student = new Student( name: "Mike");
Teacher teacher = new Teacher( name: "Jim");
Book book1 = new Book( title: "JAVA GUIDE", Item.Status.IN_STORE,
        ISBN: "978-3-16-148410-0",  author: "Kim",  genre: "guide");
Book book2 = new Book( title: "JAVA GUIDE2", Item.Status.LOST,
        ISBN: "978-1-861-97271-2",  author: "Kim",  genre: "guide");
DVD dvd1 = new DVD( title: "FNF", Item.Status.BORROWED,  director: "Mark",  minutes: 78);
DVD dvd2 = new DVD( title: "FNF2", Item.Status.IN_STORE,  director: "Mark",  minutes: 90);
Magazine magazine1 = new Magazine( title: "BAt", Item.Status.LOST,  issueNumber: 1,  publisher: "fell");
Magazine magazine2 = new Magazine( title: "BAt2", Item.Status.BORROWED,  issueNumber: 2,  publisher: "fell");

Library.borrow(student, Library.searchByTitleBorrowable("JAVA GUIDE"));
Library.borrow(teacher, Library.searchByTitleBorrowable("FNF2"));

System.out.println(student.getBorrowedItems());
System.out.println(teacher.getBorrowedItems());
```

```
[Book(super=Item(id=0001, title=JAVA GUIDE, status=BORROWED), ISBN=978-3-16-148410-0, author=Kim, genre=guide)]
[DVD(super=Item(id=0004, title=FNF2, status=BORROWED), director=Mark, minutes=90)]
```

3. The system checks:
    a. Borrowing limit
    b. User permissions (e.g., student restriction)
4. Item is marked as BORROWED and added to the user

## 2.3 Returning Items

To return an item:

```java
Student student = new Student( name: "Mike");
Teacher teacher = new Teacher( name: "Jim");
Book book1 = new Book( title: "JAVA GUIDE", Item.Status.IN_STORE,
        ISBN: "978-3-16-148410-0",  author: "Kim",  genre: "guide");
Book book2 = new Book( title: "JAVA GUIDE2", Item.Status.LOST,
        ISBN: "978-1-861-97271-2",  author: "Kim",  genre: "guide");
DVD dvd1 = new DVD( title: "FNF", Item.Status.BORROWED,  director: "Mark",  minutes: 78);
DVD dvd2 = new DVD( title: "FNF2", Item.Status.IN_STORE,  director: "Mark",  minutes: 90);
Magazine magazine1 = new Magazine( title: "BAt", Item.Status.LOST,  issueNumber: 1,  publisher: "fell");
Magazine magazine2 = new Magazine( title: "BAt2", Item.Status.BORROWED,  issueNumber: 2,  publisher: "fell");

Library.borrow(student, Library.searchByTitleBorrowable("JAVA GUIDE"));
Library.borrow(teacher, Library.searchByTitleBorrowable("FNF2"));
Library.returnItem(student, student.getBorrowedItems().getFirst());
Library.returnItem(teacher, teacher.getBorrowedItems().getFirst());

System.out.println(student.getBorrowedItems());
System.out.println(teacher.getBorrowedItems());
```

```
[]
[]
```

1. Select an item from borrowed list
2. System validates ownership
3. Item status changes to IN_STORE
4. Item is removed from user list

## 2.4 Searching Items

Users can search by:

- Title
- Author (Books)
- Director (DVDs)
- Publisher (Magazines)

Only items with status IN_STORE are returned.

## 2.5 Admin Features

Admins can:

- Generate reports of all items grouped by status:

```java
Admin admin = new Admin( name: "Rick");
Book book1 = new Book( title: "JAVA GUIDE", Item.Status.IN_STORE,
        ISBN: "978-3-16-148410-0",  author: "Kim",  genre: "guide");
Book book2 = new Book( title: "JAVA GUIDE2", Item.Status.LOST,
        ISBN: "978-1-861-97271-2",  author: "Kim",  genre: "guide");
DVD dvd1 = new DVD( title: "FNF", Item.Status.BORROWED,  director: "Mark",  minutes: 78);
DVD dvd2 = new DVD( title: "FNF2", Item.Status.IN_STORE,  director: "Mark",  minutes: 90);
Magazine magazine1 = new Magazine( title: "BAt", Item.Status.LOST,  issueNumber: 1,  publisher: "fell");
Magazine magazine2 = new Magazine( title: "BAt2", Item.Status.BORROWED,  issueNumber: 2,  publisher: "fell");
admin.reportItems();
```

```
In Store Items [DVD(super=Item(id=0004, title=FNF2, status=IN_STORE), director=Mark, minutes=90), Book(super=Item(id=0001, title=JAVA GUIDE, status=IN_STORE), ISBN=978-3-16-148410-0, author=Kim, genre=guide)]

Borrowed Items: [Magazine(super=Item(id=0006, title=BAt2, status=BORROWED), issueNumber=2, publisher=fell), DVD(super=Item(id=0003, title=FNF, status=BORROWED), director=Mark, minutes=78)]

Lost Items [Magazine(super=Item(id=0005, title=BAt, status=LOST), issueNumber=1, publisher=fell), Book(super=Item(id=0002, title=JAVA GUIDE2, status=LOST), ISBN=978-1-861-97271-2, author=Kim, genre=guide)]
```

  - IN_STORE
  - BORROWED
  - LOST
- Backup all users and items to CSV files

### 2.6 Data Storage

The system automatically:

- Loads data from CSV files on startup
- Saves data using backup functions

## 3. Developer Guide

### 3.1 Project Structure

- `domain/` → Core business logic (User, Item, Library)
- `interfaces/` → Interfaces like Reportable
- `util/` → Constants and configuration

### 3.2 Key Design Rules

#### Add New User Types

To add a new user type:

1. Extend `User`
2. Implement `save()`
3. Define borrowing rules if needed

### Add New Item Types

To add a new item type:

1. Extend `Item`
2. Implement `save()`
3. Add fields and constructors
4. Update `Library.initItems()` switch statement

## 3.3 CSV File Format

### Items CSV Format

`Type,Id,Title,Status,ExtraFields...`

```
Magazine,0006,BAt2,BORROWED,2,fell
DVD,0004,FNF2,IN_STORE,Mark,90
Magazine,0005,BAt,LOST,1,fell
Book,0002,JAVA GUIDE2,LOST,978-1-861-97271-2,Kim,guide
DVD,0003,FNF,BORROWED,Mark,78
Book,0001,JAVA GUIDE,IN_STORE,978-3-16-148410-0,Kim,guide
```

### Users CSV Format

`Type,Id,Name,ItemIds...`

```
Admin,0001,Rick,
Student,0002,Mike,
Teacher,0003,Jim,
```

## 3.4 Important Classes

### Library

Responsible for:

- Loading CSV files
- Saving CSV files
- Borrow/return logic

- Searching items

### Item

Base class for all library items.

Contains:

- ID generation system
- Global item registry (`Map<String, Item> id:Item`)

### User

Base class for all users.

Contains:

- Borrowed item list
- ID generation
- Borrowing rules

## 3.5 Design Notes for Future Developers

### Good Practices in This Project

- Uses inheritance for extensibility
- Uses polymorphism for different behaviors
- Uses enums for item status
- Uses comparators for sorting flexibility

### Known Limitations

- No database integration (CSV only)
- No concurrency control
- No user authentication system
- File writing appends instead of overwriting (may duplicate data)

## 3.6 Extension Guidelines

When extending the system:

- Do NOT modify `Item` or `User` directly unless necessary

- Prefer extending classes or adding services
- Keep business logic inside `Library`
- Maintain consistency with CSV formats

## 4. Conclusion

This system is designed to be both functional and extensible while allowing future developers to easily expand functionality, such as new item types or user roles.